Task-1

Vulnerability Assessment For A Live Website

Target: OWASP Juice Shop
Prepared by: Sai Manikanta
Tools: OWASP ZAP
Date: [28/3/2026]

Introduction

Introduction:

This report presents the findings of a vulnerability assessment conducted on OWASP juice shop using OWASP ZAP and Nmap.The puposen of this assessment is to identify potential security weakness inthe application and provide recommendation to improve its security posture.

Scope:

- Public pages only
- Passive scanning techniques
- No exploitation or active attacks performed

Tools Used:

- **OWASP-ZAP-**fofr web vulnerability scanning.
- **Kali Linux-**testing environment.
- **Nmap-**for network and port scanning

Methodology:

- Passive scanning of web application traffic
- Analysis of HTTP headers and cookies
- Identification of misconfigurations
- Basic port scanning for exposed services

Port Scanning Using (Nmap)

Objective:

- Identify open ports and Services running on the target system.

Tools Used:

- Nmap

Command Used:

- nmap -sV juiceshop.com

Findings:

- Multiple open ports were detected
- Services associated with these ports were identified

Impact:

- Open ports may expose vulnerable services
- Open ports increase the attack surface

Solution:

- Close unnecessary ports
- Use firewall rules to restict access
- Keep services updated and secured

```
└─$ nmap -sV nmap.org
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-28 14:13 IST
Nmap scan report for nmap.org (50.116.1.184)
Host is up (0.011s latency).
Other addresses for nmap.org (not scanned): 2600:3c01:e000:3e6::6d4e:7061
rDNS record for 50.116.1.184: ack.nmap.org
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      OpenSSH 7.4 (protocol 2.0)
80/tcp  open  http     Apache httpd 2.4.6
443/tcp open  ssl/http Apache httpd 2.4.6
Service Info: Host: issues.nmap.org

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 114.97 seconds
```

1.Cross-Domain Misconfiguration (CORS)

Risk:

- Medium

Description:

- The application allows cross-origin requests from any domain using the wildcard (*) in the Access-Control-Allow-Origin header

Impact:

- Sensitive data mabe exposedto unauthorised third parties
- Increase risk of data leakage and API abuse

Solution:

- Restricted access to specific trusted domains
- Avoid using wildcard (*) in CORS configuration
- Implementation proper access control policies

Result Screenshot:

Cross-Domain Misconfiguration

URL: https://juiceshop.com/cdn/shop/t/45/assets/fancybox.css?v=30466120580444283401765479840
Risk: Medium
Confidence: Medium
Parameter:
Attack:
Evidence: access-control-allow-origin: *
CWE ID: 264
WASC ID: 14
Source: Passive (10098 - Cross-Domain Misconfiguration)
Input Vector:

Description:

Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

Other Info:

The CORS misconfiguration on the web server permits cross-domain read requests from arbitrary third party domains, using unauthenticated APIs on this domain. Web browser implementations do not permit arbitrary third partie
response from authenticated APIs, however. This reduces the risk somewhat. This misconfiguration could be used by an attacker to access data that is available in an unauthenticated manner, but which uses some other form of sec
address white-listing.

Solution:

Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance).
Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner.

2.CSP Misconfiguration (style-src Unsafe-inline)

Risk:

- Medium

Discription:

- The content security policy allows the use of inline styles trough ‘unsafe-inline’
- This weakness the protection mechanism provided by CSP

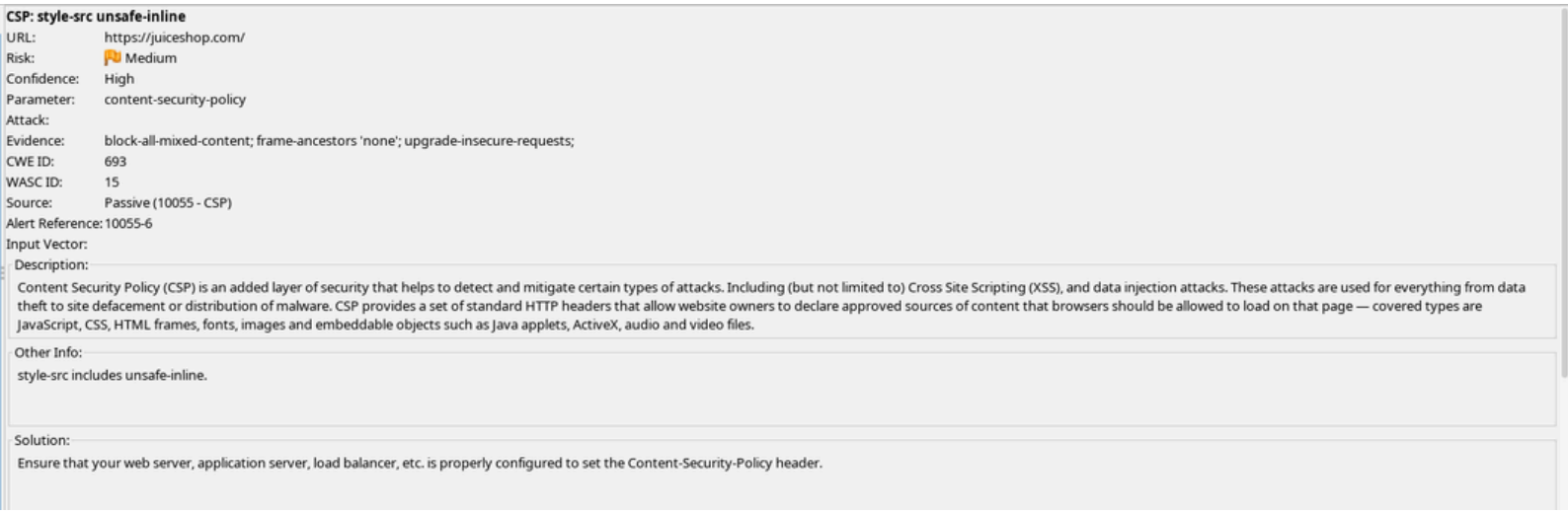
Impact:

- Attackers may inject malcious scripts or styles
- Increased risk of Cross-Site Scripting (XSS) attacks
- May lead to data theft or unauthorised actions

Solution:

- remove ‘unsafe-inline’ from CSP directives
- Use hashes or nonces for inline content
- Define a strict content security policy

Result Screenshot:



CSP: style-src unsafe-inline

URL:	https://juiceshop.com/
Risk:	Medium
Confidence:	High
Parameter:	content-security-policy
Attack:	
Evidence:	block-all-mixed-content; frame-ancestors 'none'; upgrade-insecure-requests;
CWE ID:	693
WASC ID:	15
Source:	Passive (10055 - CSP)
Alert Reference:	10055-6
Input Vector:	

Description:

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks. Including (but not limited to) Cross Site Scripting (XSS), and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.

Other Info:

style-src includes unsafe-inline.

Solution:

Ensure that your web server, application server, load balancer, etc. is properly configured to set the Content-Security-Policy header.

3.Cookie Without Secure Flag

Risk:

- Low

Discription:

- The application sets cookies without the secure flag
- These coockies can be transmitted over unencrypted HTTP connecitons.

Impact:

- Attacks on the same network can intercept cookies
- May lead to session hijacking or unauthorized access

Solution:

- Enable the secure fleg for all sensitive cookies
- Ensure cookies are transmitted only over HTTPS

Result Screenshot:

Cookie Without Secure Flag

URL: https://juiceshop.com/
Risk: Low
Confidence: Medium
Parameter: localization
Attack:
Evidence: set-cookie: localization
CWE ID: 614
WASC ID: 13
Source: Passive (10011 - Cookie Without Secure Flag)
Input Vector:

Description:

A cookie has been set without the secure flag, which means that the cookie can be accessed via unencrypted connections.

Other Info:

Solution:

Whenever a cookie contains sensitive information or is a session token, then it should always be passed using an encrypted channel. Ensure that the secure flag is set for cookies containing such sensitive information.

Coclusion

The vulnerbility assessment identified several security misconfiguration in the application, primarly related related to CORS policy, Content Security policy, and cookie security.

Additional, port scanning revealed exposed servies that may increase the attack surface of the system.

Although no active axploination was performed, these issues could be leverages by attackers if not properly addressedd.it is remmended to implement secure configuratons,enforce strict access controls,and conduct regular security assessments.